

Squads Nonce-Guard

Security Assessment

May 26th, 2026 — Prepared by OtterSec

Tamta Topuria tamta@osec.io

Artem Astakhov aikasimirov@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

General Findings **4**

OS-SNG-SUG-00 | Extra Authorities May Bypass Nonce-Guard 5

Appendices

Vulnerability Rating Scale **6**

Procedure **7**

© 2026 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Squads engaged OtterSec to assess the `nonce-guard` program. This assessment was conducted between April 12th and April 15th, 2026. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 1 finding throughout this audit engagement.

We advised owners to avoid configuring extra authorities on protected asset accounts as the nonce-guard only secures operations that flow through the guarded account (OS-SNG-SUG-00).

© 2026 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/Squads-Protocol/nonce-guard. This audit was performed against commit 3598067.

A brief description of the programs is as follows:

Name	Description
nonce-guard	A Solana/Pinocchio PDA execution proxy allows an owner to sign once, after which the program executes supplied CPI instructions using the owner-derived guarded PDA as the signer. Its primary protection is rejecting durable-nonce transactions, preventing guarded PDA authorizations from being retained and executed later.

© 2026 Otter Audits LLC. All Rights Reserved.

03 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-SNG-SUG-00	Nonce-guard only secures operations that flow through the `guarded_account` . Separate delegates or close authorities may still act outside that path.

© 2026 Otter Audits LLC. All Rights Reserved.

Extra Authorities May Bypass Nonce-Guard OS-SNG-SUG-00

Description

Nonce-guard only protects actions routed through the `guarded_account`. Users should treat the `guarded_account` as the only trusted authority path for protected assets. Any extra delegate may bypass nonce-guard's execution policy. For a protected token account, setting another delegate may let that delegate transfer tokens without going through the guarded PDA. Similarly, a separate `close_authority` may be able to close the token account and redirect its lamports.

Remediation

Owners should avoid configuring extra authorities on protected asset accounts.

© 2026 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.